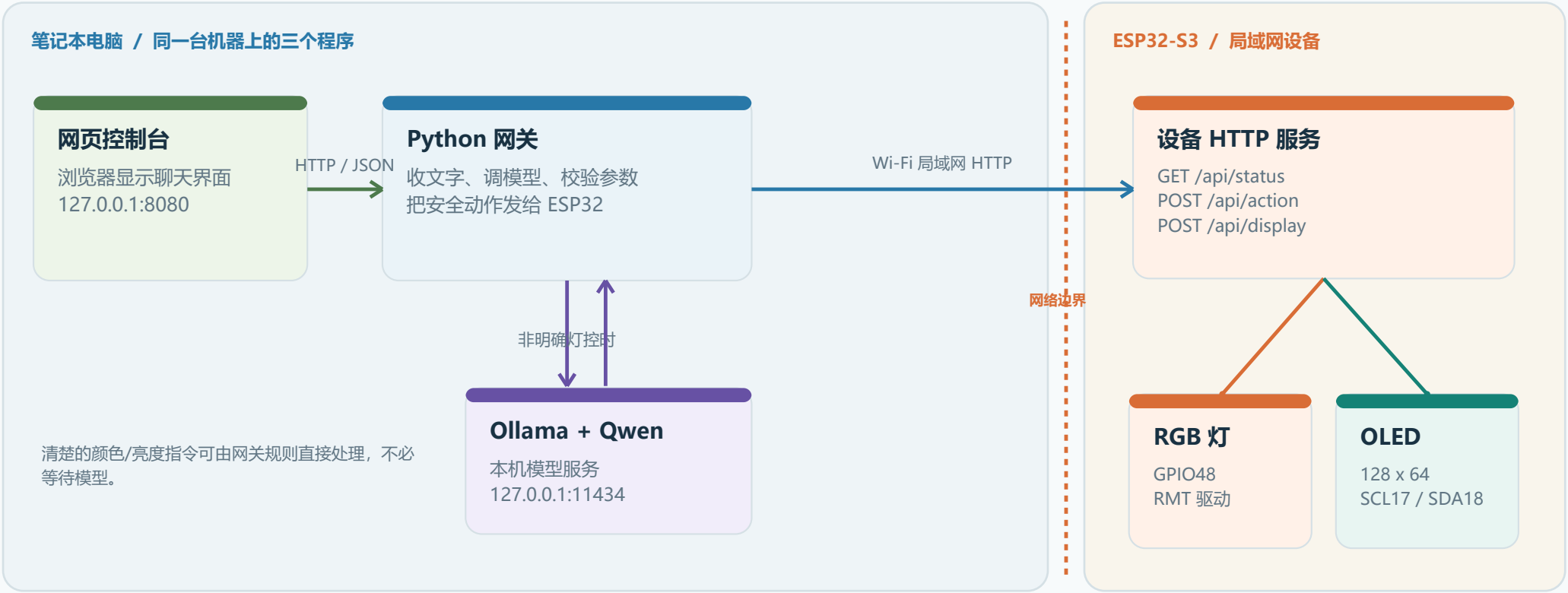
PROJECT MAP / ESP32-S3 + LOCAL LLM

从一句话到一盏灯

你的“ESP32 控制台”不是大模型装进了开发板，而是笔记本把网页、模型和设备 API 接在一起。



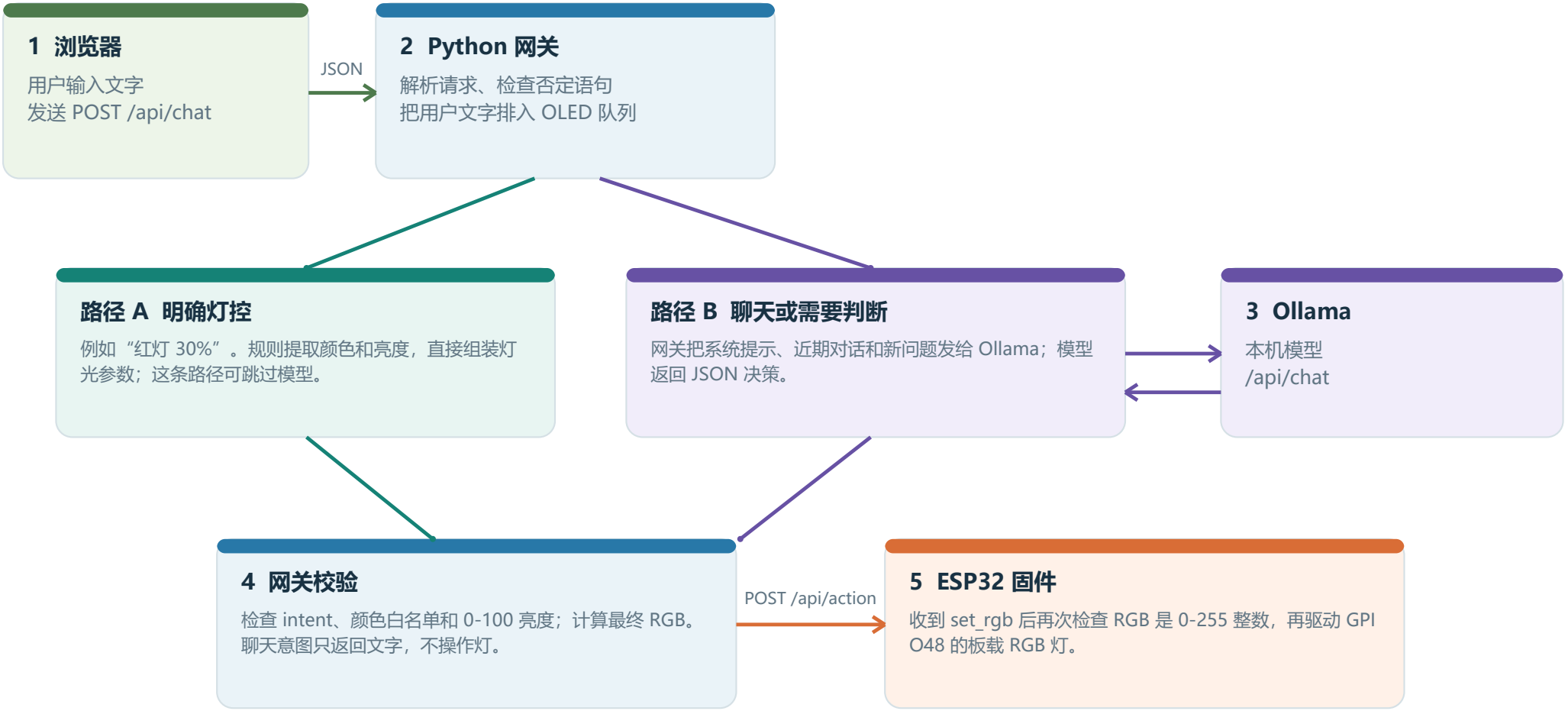
最重要的一点

大模型只在笔记本上返回文字和受限的 JSON 意图。它不直接连 GPIO，也不知道 GPIO48 是什么。Python 网关把经过检查的意图翻译成 ESP32 能执行的 HTTP 指令，ESP32 固件才真正驱动灯和屏幕。

01 / MESSAGE FLOW

发出一句话后，程序按什么顺序工作?

控制台的聊天和硬件动作共用入口，但网关会先判断是否需要调用模型。



6 ESP32 回 JSON 确认 -> 网关把 reply/action/device 返回浏览器 -> 网页更新对话和设备状态。每次输入和每次回复也会依次进入 OLED 显示队列。

大模型为什么能看懂这句话?

它不是在 ESP32 上运行，也不是有意识地理解；它根据训练得到的语言模式，在当前上下文中生成最合适的后续内容。

A 输入上下文

系统提示：限定角色、颜色和 JSON 格式
近期对话：最多保留 12 条消息
用户文字：本轮要回答的问题

B 编码成 token

分词器把文字转换为模型内部 token 序列。token 不等于固定的一个汉字或一个词。

C 模型逐步生成

模型根据上下文计算下一个 token 的概率，然后接着生成后续 token，直到完成 JSON。

D 结构化结果

Ollama 用 JSON Schema 限定字段和类型；网关再解析并做自己的校验。

当前模型输出的意思示例

```
{"intent":"set_light",
 "colors":["red","green"],
 "brightness":30,
 "reply":"mixed color, brightness 30%"}
```

intent 是程序用来分流的类别；colors 和 brightness 是控制参数；reply 是显示给人的自然语言。代码里 `DECISION_SCHEMA` 要求这些字段存在，并限制可选值。

为什么它能分类？训练阶段读过大量文本后，模型参数学到词句之间的统计关系。收到“灯调暗到 30%”时，它会利用“灯、调暗、30%”和系统提示，把输入映射为适合当前任务的字段。它可能判断错，所以系统不能把模型文本直接当硬件指令。

防止误解

“模型返回 JSON”不代表它百分之百正确。Schema 只能帮助格式受控，不能保证语义永远正确。当前程序还有显式规则、枚举白名单、范围检查、否定命令拦截和 ESP32 再校验，组成多层防线。

模型的判断怎样变成灯光?

模型不发 GPIO 电平。网关把允许的意图编译成小而明确的动作消息，固件再把动作映射到驱动函数。

1 颜色表

red = (255, 0, 0)
green = (0, 255, 0)
blue = (0, 0, 255)

2 混色与缩放

红 + 绿 = (255,255,0)
30% -> (77,77,0)
每个通道限制在 255

3 HTTP 动作

{"action":"set_rgb",
"r":77,"g":77,"b":0}

4 固件与灯

检查整数范围
RGB_BRIGHTNESS_LIMIT=48
led_strip_refresh(GPIO48)

完整的控制闭环

1. 网关请求 ESP32 的 POST /api/action。 2. 固件确认 action=set_rgb 且 r/g/b 都是 0-255 整数。 3. 固件把输入再乘以硬件亮度上限 48/255，通过 led_strip 设置单颗 RGB 灯。 4. ESP32 返回 ok 和当前逻辑 RGB。 5. 网关把结果送回网页。

单颗 RGB 灯的混色

红色加绿色的光相加会显示为黄色系，并不是两个独立的小灯。若要显示两个分开的颜色，需要多颗 LED 或灯带。

为什么用网关做校验?

把模型输出限制在预先设计的动作集合里。模型不能创建任意网络请求，也不能指定任意 GPIO。硬件动作参数由网关白名单和固件范围检查共同约束。

“ESP32 控制台”其实是网页 + 本地服务

它看起来像聊天应用，但背后是浏览器前端、笔记本 Python HTTP 服务、Ollama 和 ESP32 的分工。



这三个地址不是同一台服务

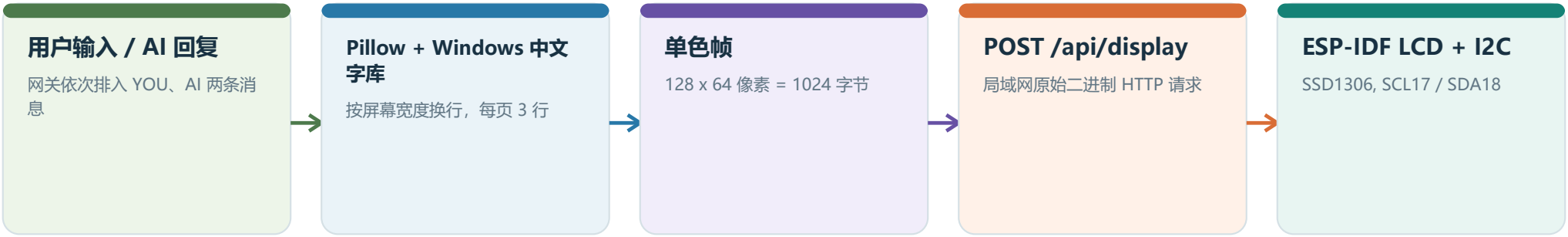
`http://127.0.0.1:8080/`	笔记本上的网页控制台
`http://127.0.0.1:11434/api/chat`	笔记本上的 Ollama 模型接口
`http://ESP32_IP/api/action`	ESP32 在 Wi-Fi 局域网里的设备接口

控制台常用路由

`GET /`	返回 index.html 网页	`GET /api/device`	读取灯和 OLED 状态
`GET /api/config`	把网关保存的设备地址交给网页	`POST /api/chat`	发送文字与会话编号
`POST /api/device/config`	保存用户填入的 ESP32 IP		

OLED 显示的是画面数据，不是模型直接写屏

网页/模型链路产生文字后，网关另开一条显示队列，把文字先在笔记本绘制成黑白像素，再交给 ESP32 刷屏。



为什么在笔记本上渲染中文?

中文字库大；当前固件分区只有 1 MB。网关借用 Windows 字体把任意输入和回复绘成位图，ESP32 只需接收固定大小的 1024 字节画面，不必内置完整中文字库。

与聊天控制并行

聊天回复不会等待屏幕逐页播放。网关后台线程按队列发送帧；ESP32 的 /api/display 收满 1024 字节后，调用 esp_lcd_panel_draw_bitmap 更新 OLED。OLED 状态还可从 /api/status 的 oled_ready 查看。

OLED 线路与接口

```
SCL -> GPIO17      SDA -> GPIO18      Address: 0x3C / 0x3D
Laptop gateway -> POST http://ESP32_IP/api/display
ESP32 -> I2C bus -> SSD1306 pixels
```

以后增加语音或新硬件，按同一条规则扩展

尽量复用“输入 -> 意图/动作 -> 网关校验 -> ESP32 执行 -> 状态反馈”，不要让模型直接控制引脚。

例子：增加语音交互

麦克风录音 -> 语音识别 ASR/STT 转成文字 -> 复用现有 /api/chat -> 原有规则/模型理解和设备控制 -> 得到回复文字 -> 语音合成 TTS 播放。若麦克风接 ESP32，可先由 ESP32 采集音频，再送笔记本或语音服务识别。

新增一个硬件动作的 5 步

1. 定义用户会怎么说，以及允许的参数范围。
2. 在 JSON Schema / 系统提示中加入动作名称与字段。
3. 在网关增加白名单分支和参数校验。
4. 在 ESP32 HTTP 接口中实现动作与底层驱动。
5. 返回执行结果，更新网页 / OLED，并补测试。

新增功能	网关要做什么	ESP32 要做什么
蜂鸣器	限制频率/时长，只接受 beep 等白名单动作	加入 buzzer API，驱动有源蜂鸣器或 PWM
舵机	把角度限制在安全范围，如 0-180 度	接收 set_servo，输出舵机 PWM
语音	ASR 转文字，复用 /api/chat；可用 TTS 读回复	若麦克风接板子，则采音并把音频送网关

安全边界

ESP32 接口目前没有登录验证，只限可信 Wi-Fi 演示，不要映射到公网。模型输出先当“建议动作”，由网关白名单和固件参数范围决定是否真的执行。

REFERENCE / DEBUG MAP

遇到问题时，先看哪一层?

用接口边界定位故障：网页、网关、模型、设备网络、固件、外设逐层检查。

网页打不开

确认网关运行；打开 http://127.0.0.1:8080/。

普通聊天失败

确认 Ollama 正在运行，模型名是 qwen3.5-2b-local。

灯不动

检查笔记本和 ESP32 同一 Wi-Fi、IP 正确；看 /api/action 返回。

OLED 不显示

检查 status 的 oled_ready；看串口地址、SCL17/SDA18 和屏幕供电。

工程里的入口文件

```
host_gateway/web/index.html  ■■■ fetch ■■
host_gateway/gateway.py      ■■■Ollama■■■■■■
main/main.c                  ESP32 HTTP■RGB■OLED
main/CMakeLists.txt          ■■■■■■
```

记忆口诀：浏览器负责交互，网关负责判断与安全，模型负责语言推断，ESP32 负责实时硬件。

参考

本工程：host_gateway/gateway.py、host_gateway/web/index.html、main/main.c。

Ollama 官方文档：Chat API；Structured Outputs。

Espressif ESP-IDF v5.4 官方文档：HTTP Server、I2C Master、LCD Panel。

手机为什么不能直接打开 127.0.0.1?

关键是分清“哪台设备上的地址”与“哪个程序的端口”。手机控制台网页由笔记本提供，ESP32 只是被网关调用的设备。



手机和笔记本都打开

`http://10.238.99.63:8080/`

http:// 协议	10.238.99.63 笔记本 Wi-Fi IP	:8080 网关网页端口	/ 首页

网关启动时绑定笔记本 Wi-Fi 网卡（地址改变后要更新）：

```
python gateway.py --host 10.238.99.63 --device-url http://10.238.99.227
```

127.0.0.1

是“回到本机”地址；手机、笔记本各自有自己的 127.0.0.1。

端口号

8080 是笔记本网页网关；ESP32 的设备 API 用默认 HTTP 端口 80。

同一 Wi-Fi

手机与笔记本须在同一个可信、非访客 Wi-Fi；路由器不能启用客户端隔离。

Windows 防火墙

Public 网络策略可能拦截入站连接；只允许可信 WLAN 的 TCP 8080，不要关闭防火墙。

安全提示：控制台没有登录验证；同一 Wi-Fi 上的其他设备也可能操作 ESP32，只在可信网络中开放。

网络基础：IETF RFC 1122 loopback；Python HTTP server bind address；Windows Firewall network profiles